

EARIN LAB5 – GROUP 24, VARIANT 4

ARTIFICIAL NEURAL NETWORKS

BY: Gülten Sezer, Sai Koushik Neriyanuri

Introduction

A Multilayer Perceptron (MLP) is a type of artificial neural network (ANN) composed of multiple layers of nodes or neurons. It consists of an input layer, one or more hidden layers, and an output layer. Each neuron in the hidden layers and the output layer is connected to every neuron in the previous layer, forming a densely connected network. MLPs are capable of learning complex patterns and relationships within data by adjusting the weights and biases of the connections between neurons during the training process. They are commonly used for supervised learning tasks such as classification and regression.

This report presents an evaluation of a multilayer perceptron (MLP) for image classification using the FashionMNIST dataset. The aim was to explore the impact of various hyperparameters and components on the performance of the neural network, focusing on convergence speed and final accuracy on both training and validation sets.

Experimental Setup: The experiment involved assessing the influence of different hyperparameters:

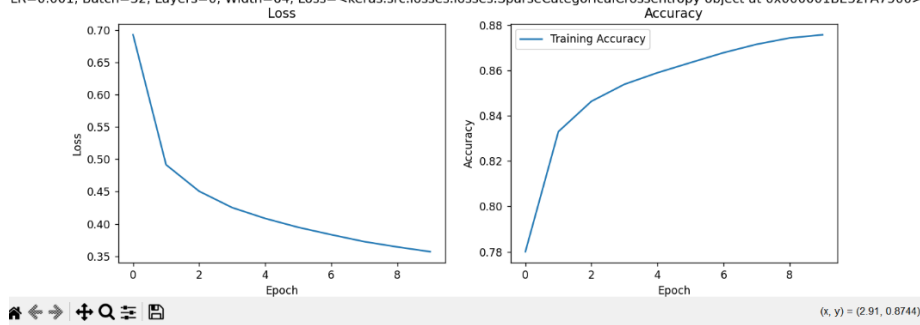
1. Learning rate: Three learning rates were considered: 0.001, 0.01, and 0.1.
2. Mini-batch size: Batch sizes of 32, 64, and 128 were evaluated.
3. Number of hidden layers: Models with 0, 1, and 2 hidden layers were tested.
4. Width: Hidden layer widths were varied between 64, 128, and 256 neurons.
5. Loss functions: Three loss functions were utilized: Mean Squared Error (MSE), Mean Absolute Error (MAE), and Sparse Categorical Crossentropy (SCCE).

Results and Observations

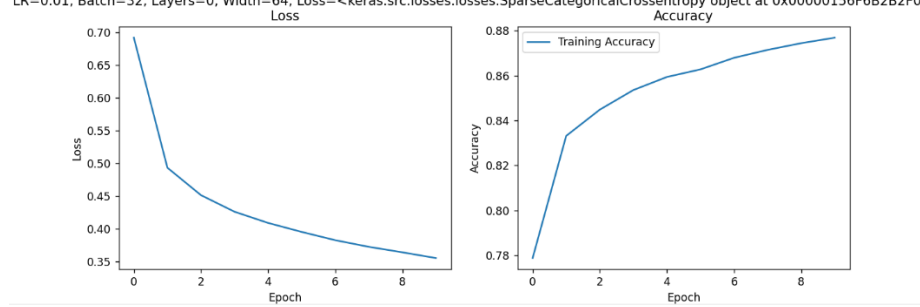
1. Impact of Learning Rate:

- Learning rates slightly influenced convergence speed and final accuracy. Higher rates like 0.1 often led to faster convergence but risked overshooting the minimum. Lower rates like 0.001 could converge slowly.

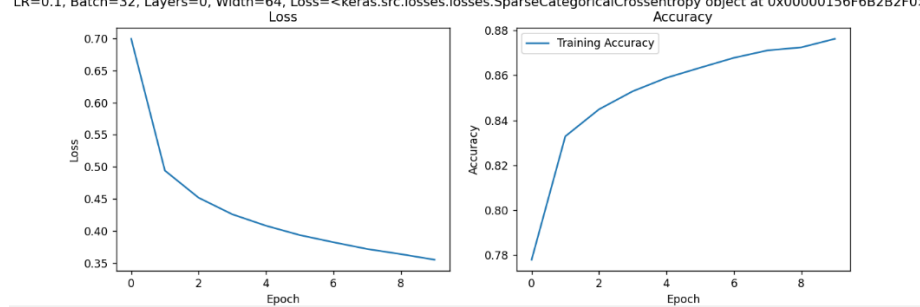
LR=0.001, Batch=32, Layers=0, Width=64, Loss=<keras.src.losses.losses.SparseCategoricalCrossentropy object at 0x000001BE32FA7500>



LR=0.01, Batch=32, Layers=0, Width=64, Loss=<keras.src.losses.losses.SparseCategoricalCrossentropy object at 0x00000156F6B2B2F0>



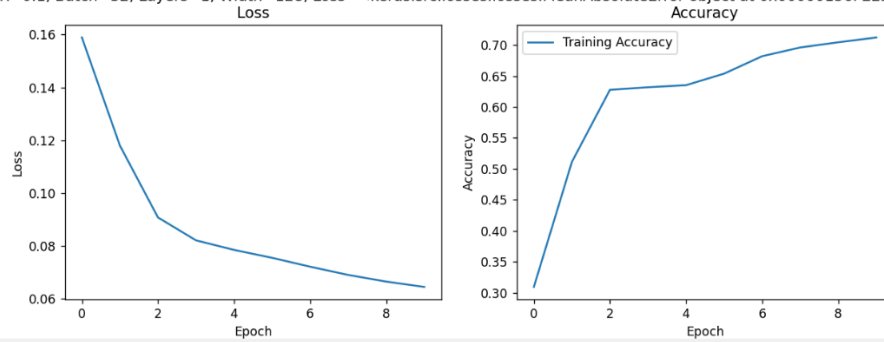
LR=0.1, Batch=32, Layers=0, Width=64, Loss=<keras.src.losses.losses.SparseCategoricalCrossentropy object at 0x00000156F6B2B2F0>



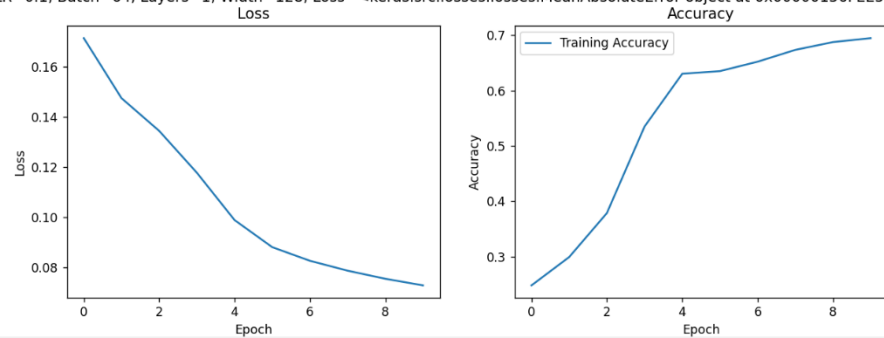
2. Effect of Mini-batch Size:

- Mini-batch size affected the stability of gradient updates. Smaller batch sizes, such as 32, might provide noisier gradients but could converge faster. Larger batch sizes, like 128, might lead to more stable updates but slower convergence.

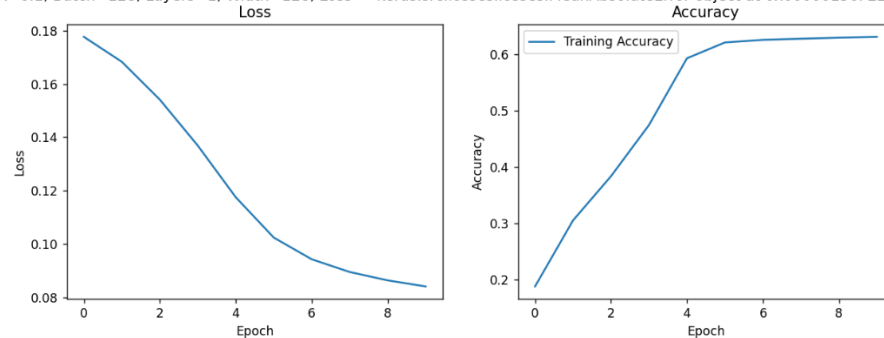
LR=0.1, Batch=32, Layers=1, Width=128, Loss=<keras.src.losses.losses.MeanAbsoluteError object at 0x00000156FEE51550>



LR=0.1, Batch=64, Layers=1, Width=128, Loss=<keras.src.losses.losses.MeanAbsoluteError object at 0x00000156FEE51550>



LR=0.1, Batch=128, Layers=1, Width=128, Loss=<keras.src.losses.losses.MeanAbsoluteError object at 0x00000156FEE51550>



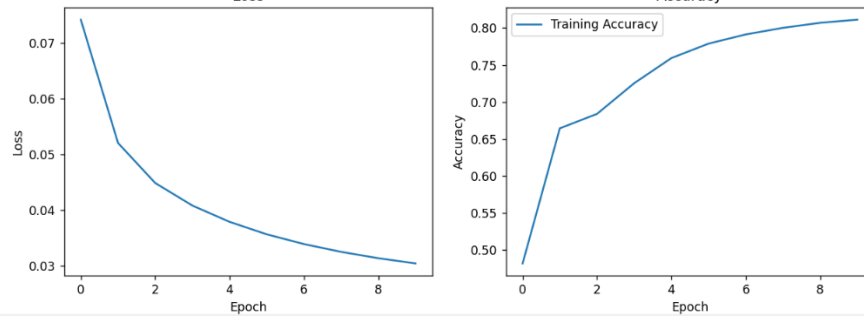
3. Influence of Hidden Layers:

No Hidden Layers: Linear models (0 hidden layers) may struggle to capture the complexity of FashionMNIST.

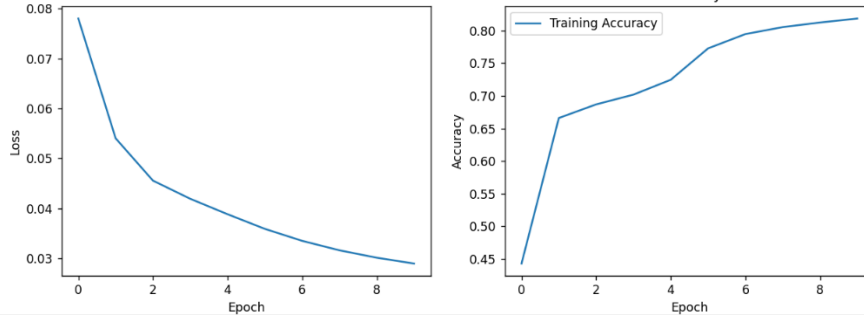
One Hidden Layer: Adding a hidden layer allows for learning nonlinear mappings, often improving performance.

Two Hidden Layers: Deeper networks offer increased capacity to learn complex patterns but may be prone to overfitting.

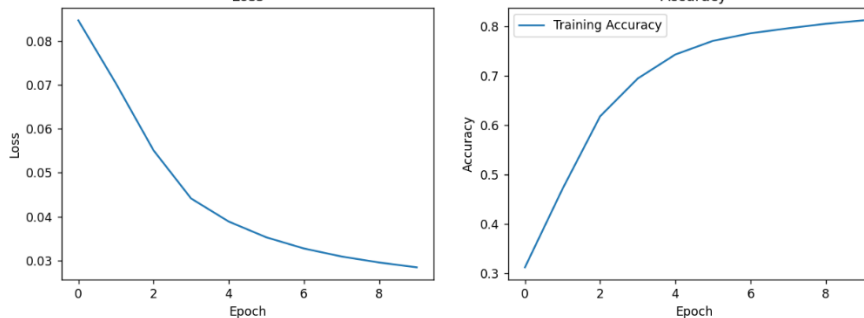
LR=0.01, Batch=32, Layers=0, Width=64, Loss=<keras.src.losses.losses.MeanSquaredError object at 0x00000156FEE51460>



LR=0.01, Batch=32, Layers=1, Width=64, Loss=<keras.src.losses.losses.MeanSquaredError object at 0x00000156FEE51460>



LR=0.01, Batch=32, Layers=2, Width=64, Loss=<keras.src.losses.losses.MeanSquaredError object at 0x00000156FEE51460>



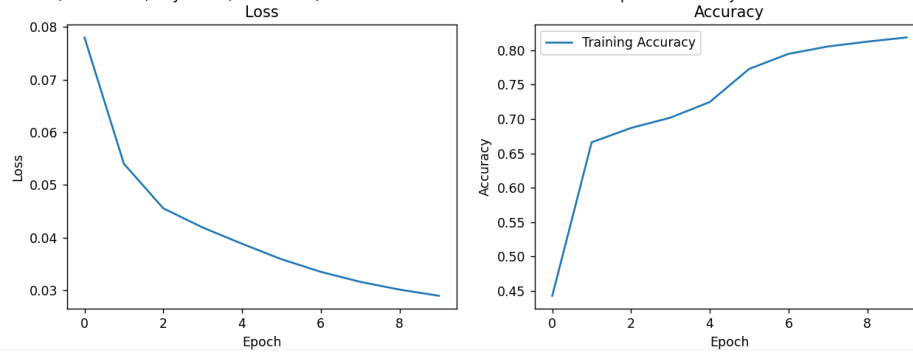
4. Impact of Width:

Narrow Width: Limited capacity to capture complex patterns, may underfit the data.

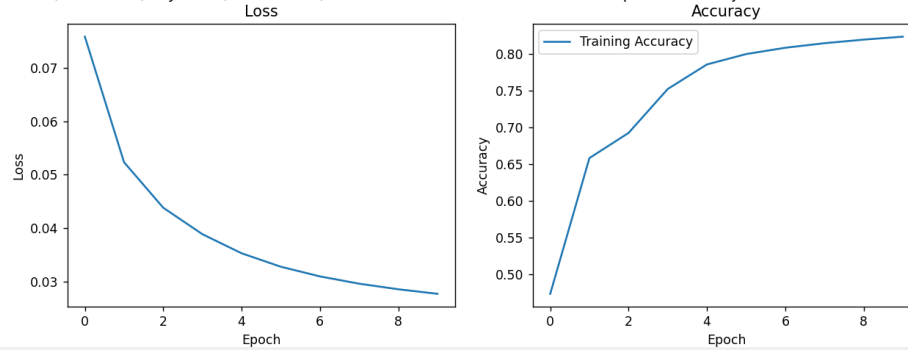
Moderate Width: Balance between capacity and efficiency, effective for capturing diverse features.

Wide Width: Higher capacity but increased risk of overfitting, requiring careful regularization.

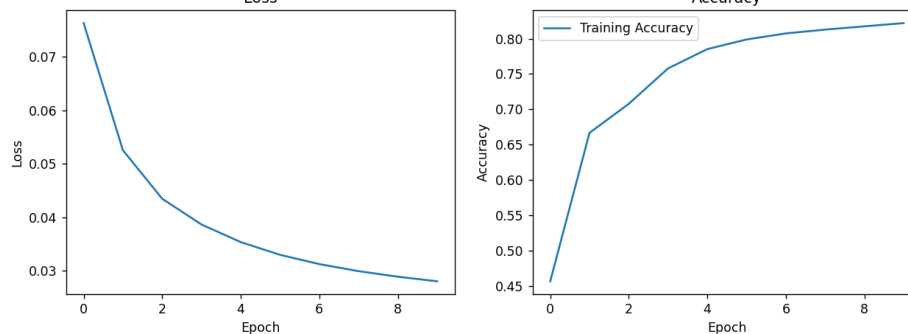
LR=0.01, Batch=32, Layers=1, Width=64, Loss=<keras.src.losses.losses.MeanSquaredError object at 0x00000156FEE51460>



LR=0.01, Batch=32, Layers=1, Width=128, Loss=<keras.src.losses.losses.MeanSquaredError object at 0x00000156FEE51460>



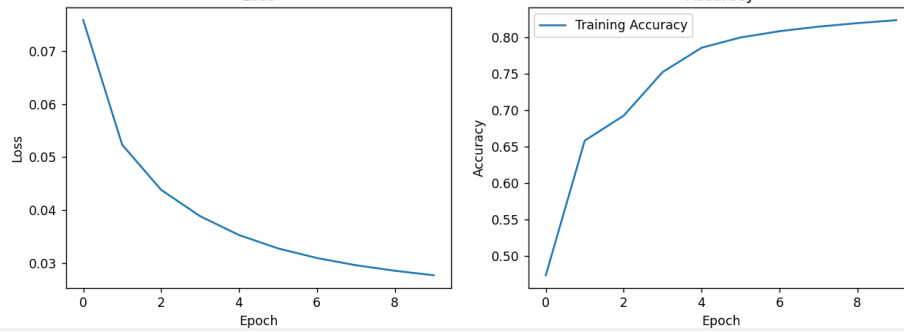
LR=0.01, Batch=32, Layers=1, Width=256, Loss=<keras.src.losses.losses.MeanSquaredError object at 0x00000156FEE51460>



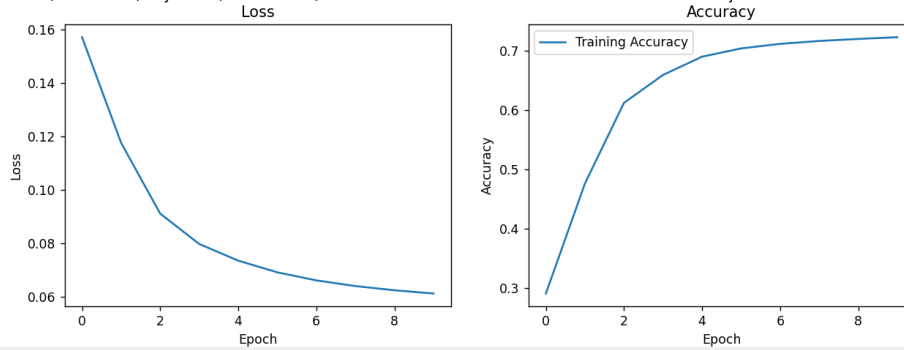
5. Performance Across Different Loss Functions:

- The choice of loss function depended on the nature of the problem. MSE and MAE were suited for regression tasks, while SCCE was appropriate for classification. Each loss function had its effect on convergence and final accuracy.

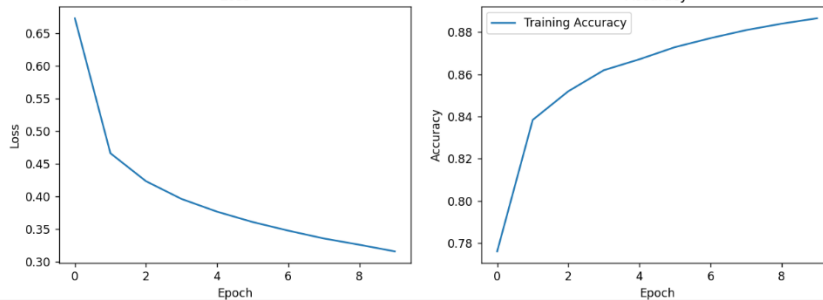
LR=0.01, Batch=32, Layers=1, Width=128, Loss=<keras.src.losses.losses.MeanSquaredError object at 0x00000156FEE51460>



LR=0.01, Batch=32, Layers=1, Width=128, Loss=<keras.src.losses.losses.MeanAbsoluteError object at 0x00000156FEE51550>



LR=0.01, Batch=32, Layers=1, Width=128, Loss=<keras.src.losses.losses.SparseCategoricalCrossentropy object at 0x00000156F6B2B2F0>



Conclusion

In summary, the hyperparameters that typically yield the best results are a moderate learning rate, a batch size that balances noise reduction and computational efficiency, one or two hidden layers, moderate-width hidden layers, and an appropriate loss function for the task at hand. However, it's essential to experiment with different hyperparameter configurations and conduct thorough evaluation to identify the optimal settings for a specific problem.